

Lab 2: Web Development Components and Security Threats

Exercise 1: Exploring Front-End Development Components

1. Identify Front-End Technologies:

Front-end technologies are tools used to create the part of a website that users see and interact with.

HTML (HyperText Markup Language)

HTML provides the basic structure and content of a webpage. It is used to create headings, paragraphs, images, links, forms, tables, and other elements.

CSS (Cascading Style Sheets)

CSS is used to style and design webpages. It controls things such as colors, fonts, spacing, borders, layouts, and how a page looks on different screen sizes.

JavaScript

JavaScript adds functionality and interactivity to webpages. It can be used for things such as buttons, forms, menus, animations, pop-ups, and updating content without refreshing the whole page.

Front-End Frameworks

Front-end frameworks provide ready-made tools and structures that make it easier to build larger and more interactive websites.

Common examples include:

- React – A popular JavaScript library for building interactive user interfaces.
- Angular – A framework used to build structured and dynamic web applications.

- Vue.js – A lightweight and flexible framework for creating user interfaces.

Summary

Technology	Main Purpose
HTML	Creates the structure and content
CSS	Controls the appearance and layout
JavaScript	Adds functionality and interaction
React	Builds interactive user interfaces
Angular	Develops structured web applications
Vue.js	Creates flexible and interactive interfaces

In simple terms: HTML builds the page, CSS makes it look good, JavaScript makes it interactive, and frameworks such as React, Angular, and Vue.js make it easier to develop complex front-end applications.

2.Create a Simple Front-End Application:

- Build a basic web application using HTML, CSS, and JavaScript that includes:
 - A header with a navigation menu.
 - A form for user input (e.g., contact form).
 - Interactive elements (e.g., a button that changes the text when clicked).

Code Used For HTML:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Simple Contact App</title>
```

```
  <link rel="stylesheet" href="styles.css">
```

```
</head>
```

```
<body>
```

```
  <header>
```

```
    <h1>Contact Us</h1>
```

```
    <nav>
```

```
      <a href="#home">Home</a>
```

```
      <a href="#about">About</a>
```

```
      <a href="#contact">Contact</a>
```

```
    </nav>
```

```
  </header>
```

```
  <main>
```

```
    <h2>Get in Touch</h2>
```

```
    <form>
```

```
      <label for="name">Name:</label>
```

```
      <input type="text" id="name" name="name" required>
```

```
      <label for="email">Email:</label>
```

```
      <input type="email" id="email" name="email" required>
```

```
      <label for="message">Message:</label>
```

```
<textarea id="message" name="message"></textarea>
```

```
<button type="submit">Submit</button>  
</form>
```

```
<br>
```

```
<button id="changeButton">Click Me</button>  
<p id="text">Click the button to change this text.</p>  
</main>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

```
un Terminal Help < -> Htmlwebsitedev1 Update
... Welcome x index.html x # styles.css JS script.js
index.html > html > C:\Users\HP\Documents\Htmlwebsitedev1\index.html
1  <!DOCTYPE html>
2
3  <html lang="en">
4
5  <head>
6      <meta charset="UTF-8">
7      <meta name="viewport" content="width=device-width, initial-scale=1.0">
8      <title>Simple Contact App</title>
9      <link rel="stylesheet" href="styles.css">
10 </head>
11
12 <body>
13
14
15     <header>
16         <h1>Contact Us</h1>
17
18         <nav>
19             <a href="#home">Home</a>
20             <a href="#about">About</a>
21             <a href="#contact">Contact</a>
22         </nav>
23     </header>
24
25     <main>
26         <h2>Get in Touch</h2>
27
28         <form>
29             <label for="name">Name:</label>
```

Code Used For CSS:

```
body {  
font-family: Arial, sans-serif;  
margin: 0;  
padding: 0;  
background-color: #f4f4f4;  
}
```

```
header {  
background-color: #333;  
color: white;  
padding: 20px;  
text-align: center;  
}
```

```
nav a {  
color: white;  
text-decoration: none;  
margin: 15px;  
}
```

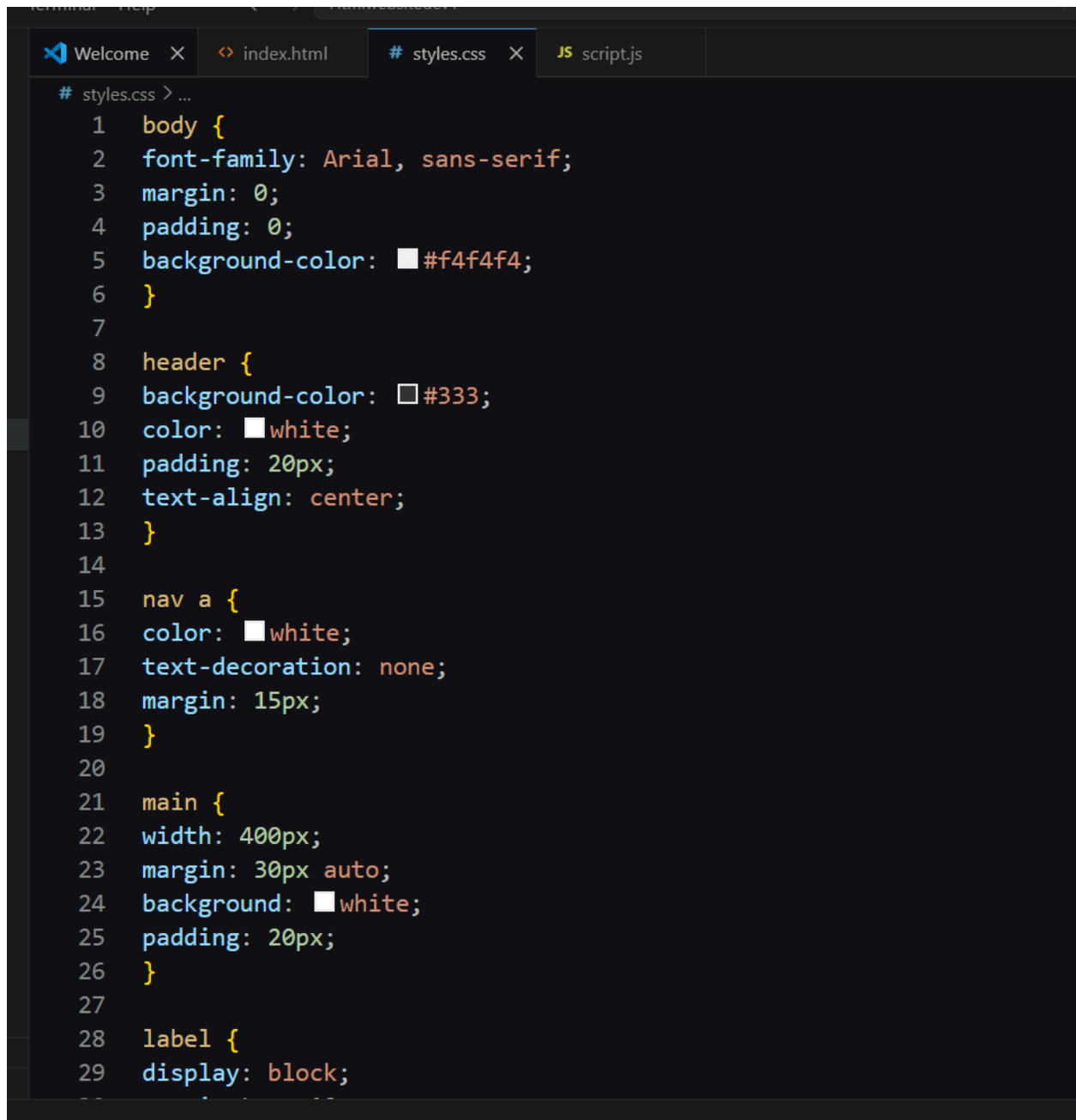
```
main {  
width: 400px;  
margin: 30px auto;  
background: white;  
padding: 20px;  
}
```

```
label {  
display: block;  
margin-top: 10px;  
}
```

```
input, textarea {  
width: 100%;  
padding: 8px;
```

```
margin-top: 5px;  
}
```

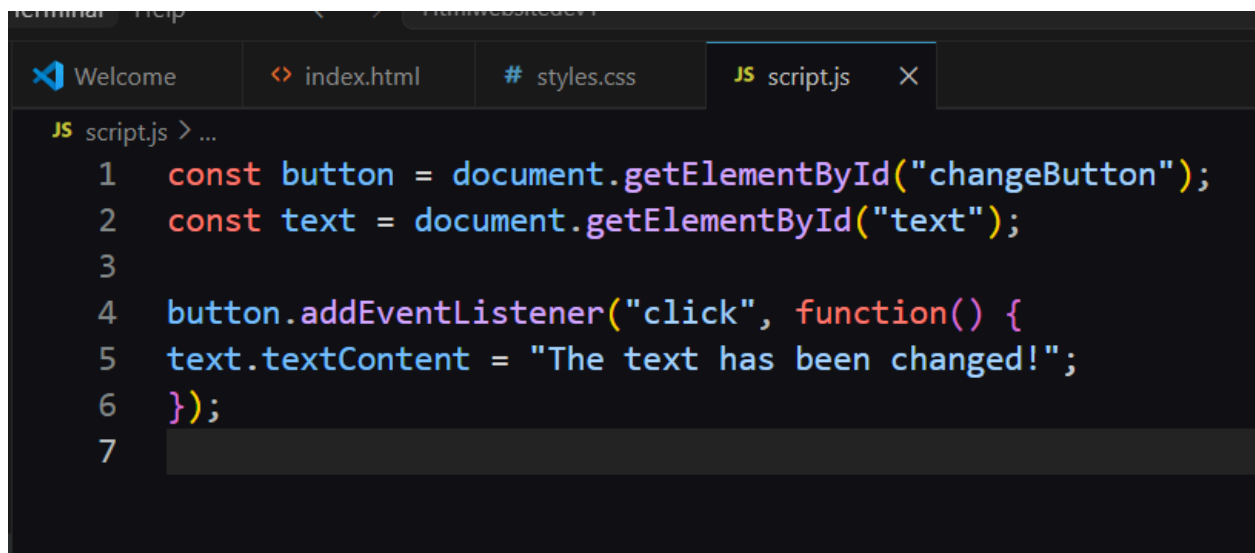
```
button {  
margin-top: 15px;  
padding: 10px 15px;  
cursor: pointer;  
}
```



Code Used For Js.

```
const button = document.getElementById("changeButton");  
const text = document.getElementById("text");
```

```
button.addEventListener("click", function() {  
text.textContent = "The text has been changed!";  
});
```

A screenshot of a code editor window with a dark theme. The editor has four tabs at the top: 'Welcome', 'index.html', 'styles.css', and 'script.js'. The 'script.js' tab is active. The code in the editor is as follows:

```
JS script.js > ...  
1  const button = document.getElementById("changeButton");  
2  const text = document.getElementById("text");  
3  
4  button.addEventListener("click", function() {  
5  text.textContent = "The text has been changed!";  
6  });  
7
```

Outcome:

The JavaScript event handler successfully changed the displayed text when the button was clicked.

The screenshot shows a web browser window with a single tab titled 'Simple Contact App'. The address bar displays the URL '127.0.0.1:5500/index.html?name=Chukwuemeka+Olivil+Onyinye&email=olivil247%40gmail.com&message=i+love+you'. The page has a dark grey header with the title 'Contact Us' in white, and navigation links for 'Home', 'About', and 'Contact'. The main content area is light grey and contains a white box with the heading 'Get in Touch'. Inside this box, there are three input fields labeled 'Name:', 'Email:', and 'Message:'. Below the 'Message:' field is a 'Submit' button. Further down is a 'Click Me' button, followed by the text 'Click the button to change this text.'

Reflection

Front-end technologies work together to create a complete and user-friendly website. HTML provides the structure, CSS controls the appearance, and JavaScript adds interaction and functionality. When these technologies are properly combined, they make the website easier and more enjoyable to use.

Poorly written front-end code can create security problems. For example, XSS can allow attackers to inject harmful scripts into a webpage, while CSRF can trick users into performing unwanted actions.

Exercise 2: Understanding Back-End Development Components

1. Identify Back-End Technologies

Back-end technologies are used to process requests, manage data, and provide services to the front end of a web application.

Server-side programming languages:

- PHP
- Python
- Java
- JavaScript (Node.js)
- C#

Databases:

- MySQL
- MongoDB
- PostgreSQL
- SQLite

Web frameworks:

- Express.js
- Django
- Laravel
- Flask
- Spring Boot

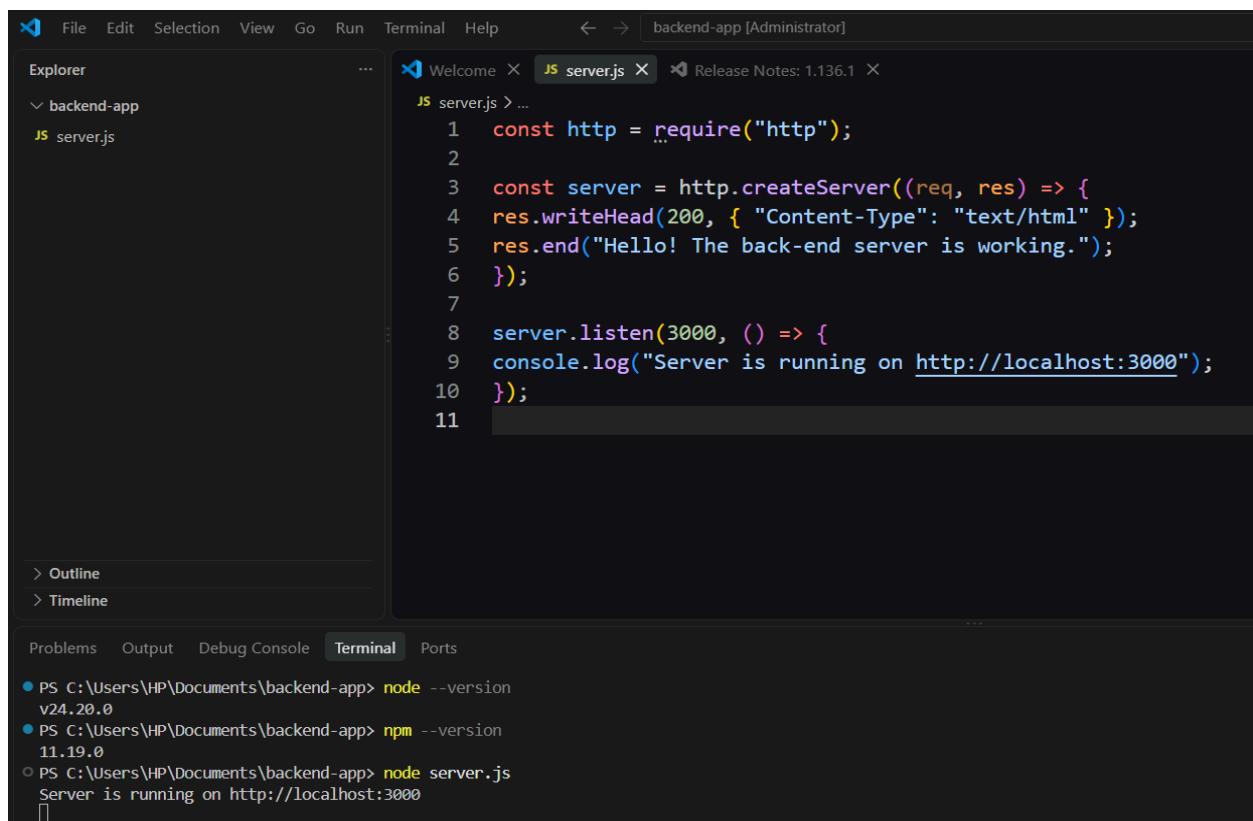
2. Created a Simple Back-End Application using Node.js

Code Used:

```
const http = require("http");
```

```
const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/html" });
  res.end("Hello! The back-end server is working.");
});

server.listen(3000, () => {
  console.log("Server is running on http://localhost:3000");
});
```

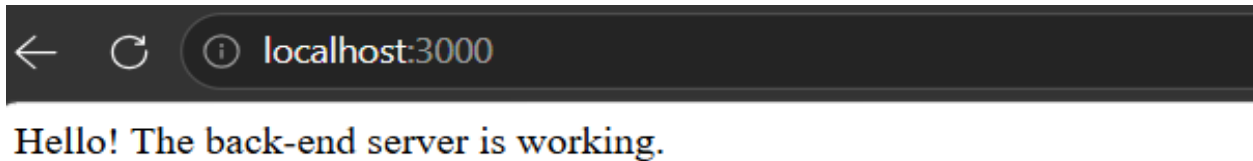
A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'backend-app' with a file 'server.js'. The main editor area displays the code for 'server.js', which is a Node.js HTTP server. The code is as follows:

```
1  const http = require("http");
2
3  const server = http.createServer((req, res) => {
4    res.writeHead(200, { "Content-Type": "text/html" });
5    res.end("Hello! The back-end server is working.");
6  });
7
8  server.listen(3000, () => {
9    console.log("Server is running on http://localhost:3000");
10 });
11
```

The bottom of the window shows the Terminal panel with the following commands and output:

```
PS C:\Users\HP\Documents\backend-app> node --version
v24.20.0
PS C:\Users\HP\Documents\backend-app> npm --version
11.19.0
PS C:\Users\HP\Documents\backend-app> node server.js
Server is running on http://localhost:3000
```

Output:



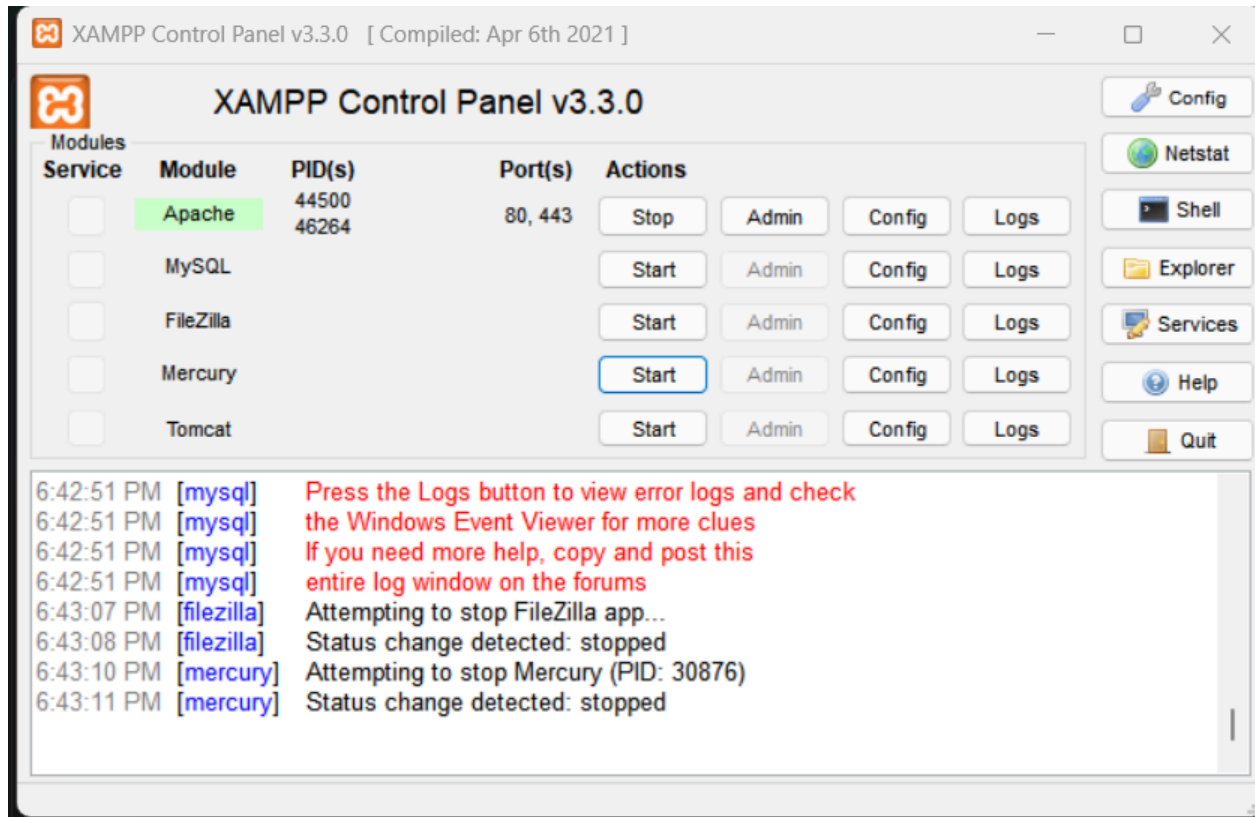
I created a basic back-end server using Node.js. The server was configured to listen on port 3000 and respond to HTTP requests. I tested it in my browser using `http://localhost:3000`, and it successfully displayed a response from the server

2.1 Created a Simple Back-End Application using PHP/MySQL

Step 1:Downloaded and Installed XAMPP

Then:

Start Apache



Apache is successfully running
also because Port 3306 is already in use from a previous MYSQL already existing on my system there was no need of starting MYSQL IN XAMPP control panel.

Step 2:On my browser I opened

<http://localhost>

Image of outcome:



XAMPP welcome/dashboard page successfully opened.

Step 3: Created database and table in mysql Workbench

The screenshot shows a SQL IDE window with a menu bar (Server, Tools, Scripting, Help) and a toolbar. The main editor displays the following SQL script:

```
1 • CREATE database userdb;
2
3 • Use userdb;
4
5 • CREATE TABLE Staffs (
6     id INT AUTO_INCREMENT PRIMARY KEY,
7     firstname VARCHAR(50),
8     lastname VARCHAR(50),
9     school VARCHAR(50)
10 );
```

Below the editor is an 'Output' panel with a dropdown menu set to 'Action Output'. It contains a table with the following data:

#	Time	Action	Message
✓ 1	19:02:11	CREATE database userdb	1 row(s) affected
✓ 2	19:09:09	Use userdb	0 row(s) affected
✓ 3	19:09:09	CREATE TABLE Staffs (id INT AUTO_INCREMENT PRIMARY KEY, firstname VARCHAR(50), lastn...	0 row(s) affected

Step 4: Create the PHP project folder In:

C:\xampp\htdocs

a new folder called:

backend-app

Then create a file inside the backend-app:

index.php

STEP 5:Opened index.php in VS Code

Open the file in VS Code

Code Use:

```
<?php
```

```
$servername = "localhost";
```

```
$username = "root";
```

```
$password = "inserted my password";
```

```
$dbname = "userdb";
```

```
$conn = new mysqli($servername, $username, $password, $dbname);
```

```
if ($conn->connect_error) {
```

```
    die("Connection failed: " . $conn->connect_error);
```

```
}
```

```
echo "PHP is connected to the database successfully!";
```

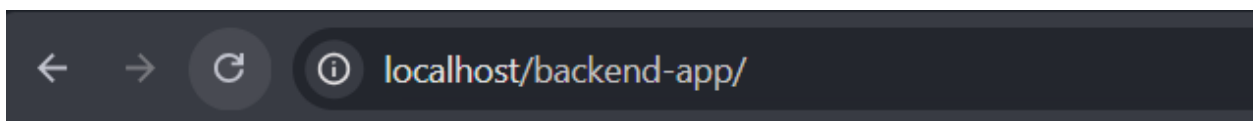
```
?>
```

C: > xampp > htdocs > backend-app > index.php

```
1  <?php
2
3  $servername = "localhost";
4  $username = "root";
5  $password = "";
6  $dbname = "userdb";
7
8  $conn = new mysqli($servername, $username, $password, $dbname);
9
10 if ($conn->connect_error) {
11     die("Connection failed: " . $conn->connect_error);
12 }
13
14 echo "PHP is connected to the database successfully!";
15
16 ?>
```

STEP 6: Opened PHP application

<http://localhost/backend-app/>



PHP is connected to the database successfully!

PHP , Apache and MySQL connection is working

STEP 7: Connected the form by

Replacing the contents of index.php with:

```
<!DOCTYPE html>
<html>
<head>
  <title>User Form</title>
</head>

<body>

  <h1>User Information</h1>

  <form method="POST">

    <label>First Name:</label>
    <input type="text" name="firstname" required>
    <br><br>

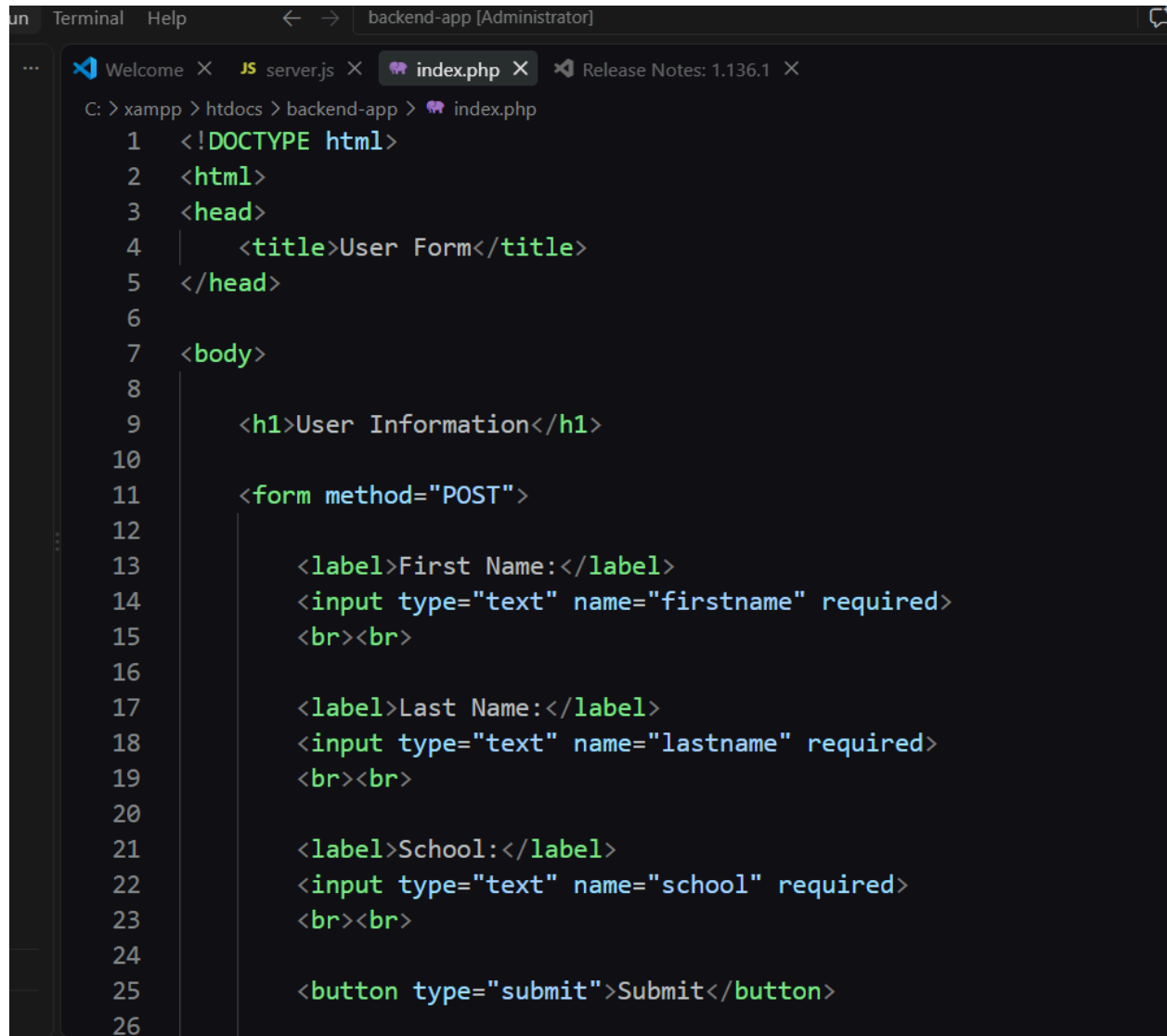
    <label>Last Name:</label>
    <input type="text" name="lastname" required>
    <br><br>

    <label>School:</label>
    <input type="text" name="school" required>
    <br><br>

    <button type="submit">Submit</button>

  </form>
```

```
</body>
</html>
```

A screenshot of a code editor window titled 'backend-app [Administrator]'. The editor shows the content of 'index.php' with line numbers 1 through 26. The code is an HTML form for user information. The form has a title 'User Form' and contains three text input fields for 'First Name', 'Last Name', and 'School', each with a 'required' attribute. A 'Submit' button is at the bottom. The code is as follows:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>User Form</title>
5  </head>
6
7  <body>
8
9      <h1>User Information</h1>
10
11     <form method="POST">
12
13         <label>First Name:</label>
14         <input type="text" name="firstname" required>
15         <br><br>
16
17         <label>Last Name:</label>
18         <input type="text" name="lastname" required>
19         <br><br>
20
21         <label>School:</label>
22         <input type="text" name="school" required>
23         <br><br>
24
25         <button type="submit">Submit</button>
26
```

Save and Refreshed:

<http://localhost/backend-app/>

← → ↻ ⓘ localhost/backend-app/

User Information

First Name:

Last Name:

School:

← → ↻ ⓘ localhost/backend-app/

User Information

First Name:

Last Name:

School:

STEP 8: Added the database code by replace the contents of index.php with:

```
<?php

$servername = "localhost";
$username = "root";
$password = "added my password";
$dbname = "userdb";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {

    $firstname = $_POST["firstname"];
    $lastname = $_POST["lastname"];
    $school = $_POST["school"];

    $sql = "INSERT INTO Staffs (firstname, lastname, school)
        VALUES ('$firstname', '$lastname', '$school')";

    if ($conn->query($sql) === TRUE) {
        echo "Information submitted successfully!";
    } else {
        echo "Error: " . $conn->error;
    }
}
```

```
}
```

```
?>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>User Form</title>
```

```
</head>
```

```
<body>
```

```
  <h1>User Information</h1>
```

```
  <form method="POST">
```

```
    <label>First Name:</label>
```

```
    <input type="text" name="firstname" required>
```

```
    <br><br>
```

```
    <label>Last Name:</label>
```

```
    <input type="text" name="lastname" required>
```

```
    <br><br>
```

```
    <label>School:</label>
```

```
    <input type="text" name="school" required>
```

```
    <br><br>
```

```
    <button type="submit">Submit</button>
```

```
  </form>
```

```
</body>
```

```
</html>
```

```
Welcome X JS server.js X index.php X Release Notes: 1.136.1 X
C: > xampp > htdocs > backend-app > index.php
1  <?php
2
3  $servername = "localhost";
4  $username = "root";
5  $password = "Princ...";
6  $dbname = "userdb";
7
8  $conn = new mysqli($servername, $username, $password, $dbname);
9
10 if ($conn->connect_error) {
11     die("Connection failed: " . $conn->connect_error);
12 }
13
14 if ($_SERVER["REQUEST_METHOD"] == "POST") {
15
16     $firstname = $_POST["firstname"];
17     $lastname = $_POST["lastname"];
18     $school = $_POST["school"];
19
20     $sql = "INSERT INTO Staffs (firstname, lastname, school)
21           VALUES ('$firstname', '$lastname', '$school')";
22
23     if ($conn->query($sql) === TRUE) {
24         echo "Information submitted successfully!";
25     } else {
26         echo "Error: " . $conn->error;
```

STEP 9 :Test the form

← → ↻ ⓘ localhost/backend-app/

User Information

First Name:

Last Name:

School:

← → ↻ ⓘ localhost/backend-app/

Information submitted successfully!

User Information

First Name:

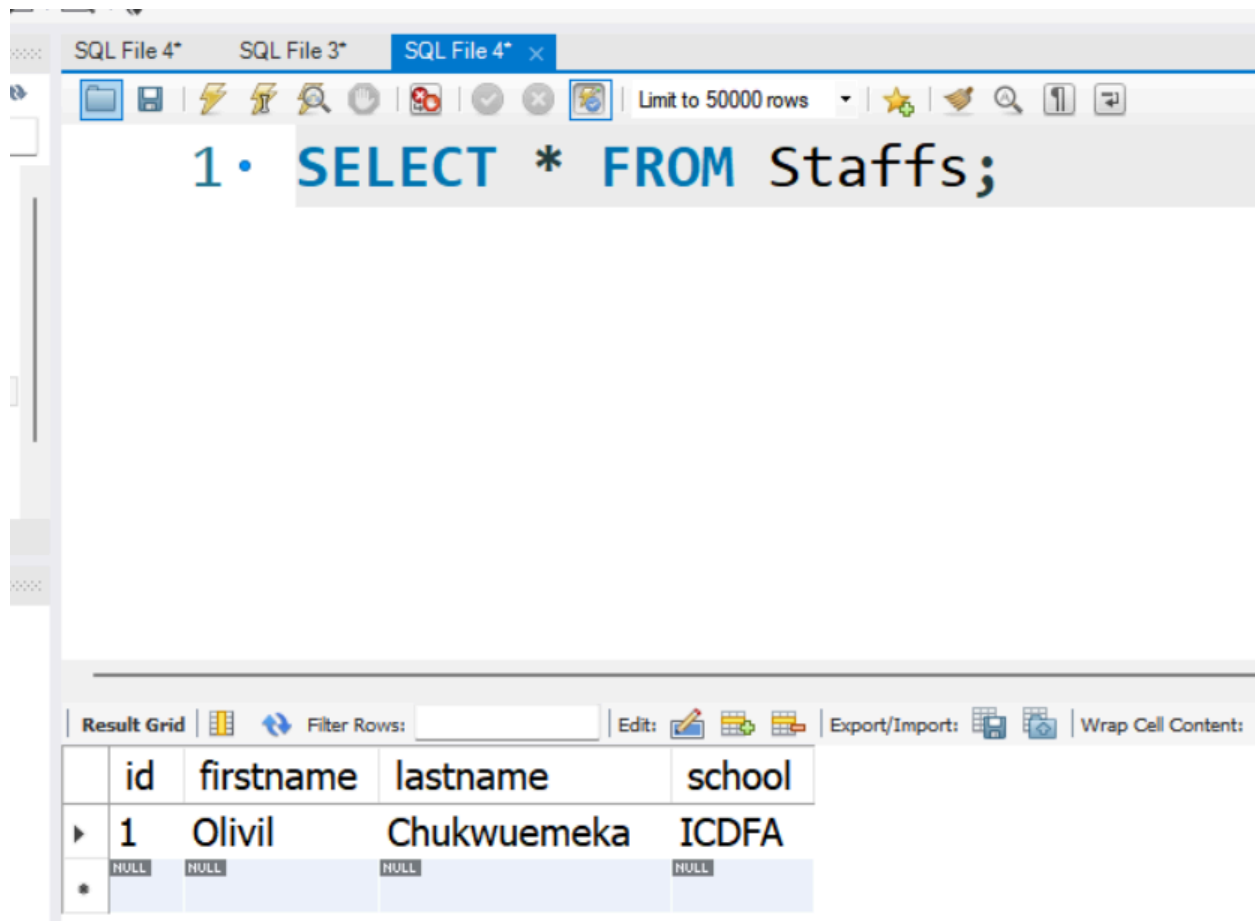
Last Name:

School:

STEP 10: On MySQL Workbench.

code:

SELECT * FROM Staffs;



Code successfully displayed the details of users Input on form.

Report:

I successfully created a simple PHP back-end application using XAMPP and connected it to MySQL database. The application accepts a user's first name, last name, and school through a form and stores the information in the Staffs table. I tested the application through localhost and confirmed that the submitted data was stored successfully in MySQL. Code analysis

identified a SQL injection vulnerability because user-supplied values are concatenated directly into the SQL statement without parameterized queries.

3.Reflection:

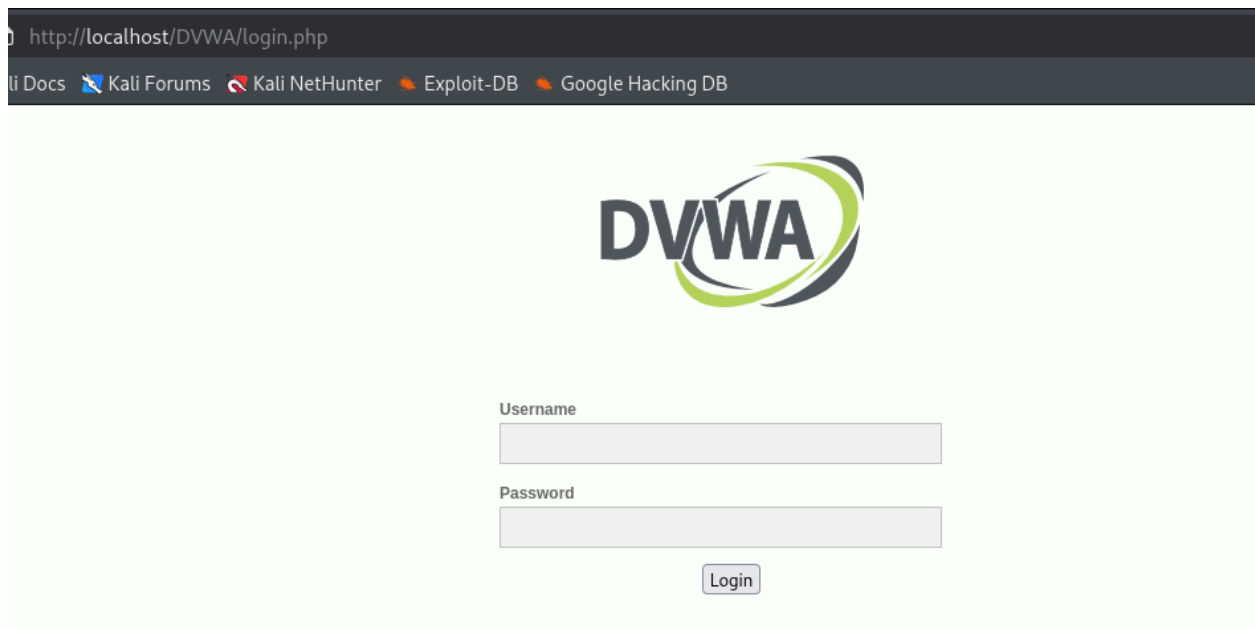
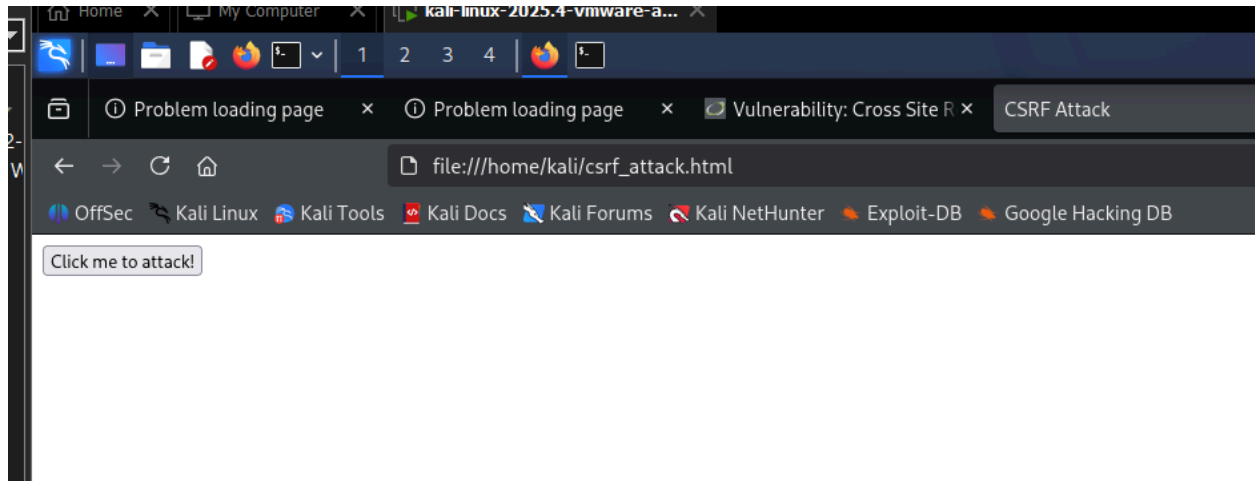
The back-end is responsible for handling data, processing requests, and connecting the application to the database. If the back-end has vulnerabilities such as SQL injection or insecure APIs, attackers can access, change, or steal sensitive information. This can put the entire application and its users at risk.

1. Common Security Threats

Five common web security threats are:

- **SQL Injection:** Attackers insert harmful SQL commands to access or manipulate database information.
- **Cross-Site Scripting (XSS):** Attackers inject malicious scripts into web pages that may affect other users.
- **Cross-Site Request Forgery (CSRF):** Tricks a logged-in user into performing an action they did not intend to perform.
- **Security Misconfiguration:** Poorly configured servers, applications, or security settings can expose sensitive information.
- **Insecure Deserialization:** Unsafe handling of serialized data can allow attackers to manipulate data or execute unwanted actions.

2. Sample Application Analysis



' OR '1'='1



I used intentionally vulnerable web applications to examine common web security weaknesses, including SQL injection, XSS, and CSRF. The documented SQL injection test demonstrated how weak input handling could result in authentication bypass.

Result: The SQL injection payload successfully bypassed the authentication control in the intentionally vulnerable laboratory application and authenticated the session as an administrator.

3. Reflection

These security threats can lead to data theft, unauthorized access, and changes to application data. Developers can reduce these risks by using input validation, prepared SQL statements, secure authentication, proper access controls, safe configuration settings, and regular security testing.

Summary of Findings

During the lab, I learned how common web vulnerabilities can affect an application. I tested a vulnerable web application for issues such as SQL injection, XSS, and CSRF and observed how poor input validation and weak security controls can expose user data. I also learned that secure coding practices, proper authentication, input validation, prepared statements, and regular security testing are important for protecting web applications.

Recommendations for Secure Coding Practices in Web Development

Developers should always validate user input and use prepared statements to prevent SQL injection. They should also properly handle user sessions, protect against XSS and CSRF, use strong authentication, and avoid storing passwords as plain text. Regular security testing, software updates, and proper server configuration can also help keep web applications secure.